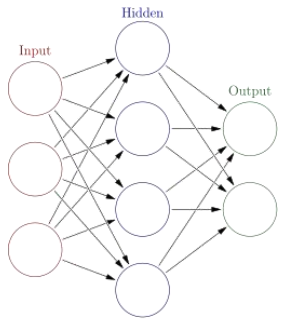
I4-AMS: Artificial Intelligence

Lecture 2: Neural Network Foundations

Neural Network



Today’s agenda

Part 1 Logistic regression as a neural network

Loss and cost, gradient descent, computation graphs, vectorization

Part 2 Shallow neural networks

Hidden layers, activation functions, backpropagation, initialization

Part 3 Deep neural networks

L-layer networks, general equations, softmax, PyTorch

Part 1

Logistic regression as a neural network

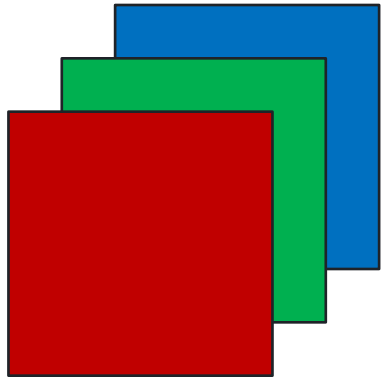


Binary classification



Question: is this a cat?

1 (Cat) Vs 0 (Non cat)



$64 \times 64 \times 3$

$$\mathbf{x} = \begin{bmatrix} 255 \\ 231 \\ 42 \\ \vdots \\ 87 \end{bmatrix} \in \mathbb{R}^{12288}$$

Label $y \in \{0, 1\}$: 1 = cat, 0 = not cat.

Unroll the three-color channels into one feature vector of length

$n = 64 \cdot 64 \cdot 3 = 12288$

Goal: learn a function $f : \mathbb{R}^n \to \{0, 1\}$ from m labelled examples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$.

Notation

Symbol	Meaning	Shape
m	number of training examples	scalar
n	number of features per example	$n = 12288$ (cat images)
$\mathbf{x}^{(i)}$	features of example i (a column vector)	$\mathbb{R}^n$
$y^{(i)}$	label of example i	$\{0, 1\}$
$\mathbf{X}$	all inputs, one example per row: row i is $\mathbf{x}^{(i)\top}$	$\mathbb{R}^{m \times n}$
$\mathbf{y}$	all labels, stacked	$\mathbb{R}^{m \times 1}$

We use the PyTorch convention: **one example per row**, so $\mathbf{X} \in \mathbb{R}^{m \times n}$.

The logistic regression model

Given $\mathbf{x} \in \mathbb{R}^n$, we want $\hat{y} = P(y = 1 \mid \mathbf{x})$: the probability that the image is a cat.

First try: a linear model

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

Problem: $\hat{y}$ can be larger than 1 or negative, so it is not a probability.

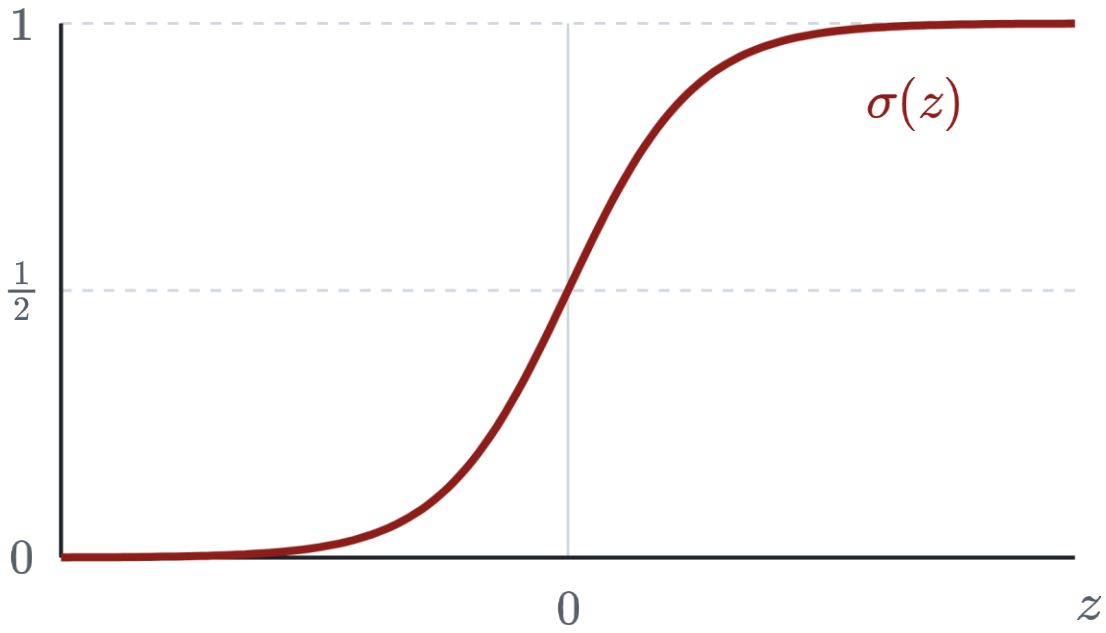
Logistic regression

$$\hat{y} = \sigma\left(\mathbf{w}^\top \mathbf{x} + b\right)$$

The sigmoid σ maps any real number into $(0, 1)$.

Parameters to learn: $\mathbf{w} \in \mathbb{R}^n$ (weights) and $b \in \mathbb{R}$ (bias). We write $z = \mathbf{w}^\top \mathbf{x} + b$ for the linear part.

The sigmoid function



$$z = \mathbf{w}^\top \mathbf{x} + b$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $z \to +\infty$: $\sigma(z) \to 1$
- $z \to -\infty$: $\sigma(z) \to 0$
- $\sigma(0) = \frac{1}{2}$
- $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$

Measuring error: the loss function

The loss $\mathcal{L}(\hat{y}, y)$ measures how wrong **one** prediction is.

Squared error?

$$\mathcal{L}(\hat{y}, y) = \tfrac{1}{2}\,(\hat{y} - y)^2$$

Combined with σ, the cost is **not convex** in $(\mathbf{w}, b)$: gradient descent can get stuck.

Binary cross-entropy

$$\mathcal{L}(\hat{y}, y) = -\big[\, y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \,\big]$$

Convex in $(\mathbf{w}, b)$ for logistic regression: a single global minimum.

If $y = 1$: $\mathcal{L}(\hat{y}, y) = -\log \hat{y}$ ← want $\log \hat{y}$ large, want $\hat{y}$ large

If $y = 0$: $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$ ← want $\log(1 - \hat{y})$ large … want $\hat{y}$ small

From loss to cost

Loss: one example

$$\mathcal{L}\big(\hat{y}^{(i)}, y^{(i)}\big)$$

Cost: the whole training set

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}\big(\hat{y}^{(i)}, y^{(i)}\big)$$

Training = solving $\min_{\mathbf{w},\, b}\ J(\mathbf{w}, b)$.

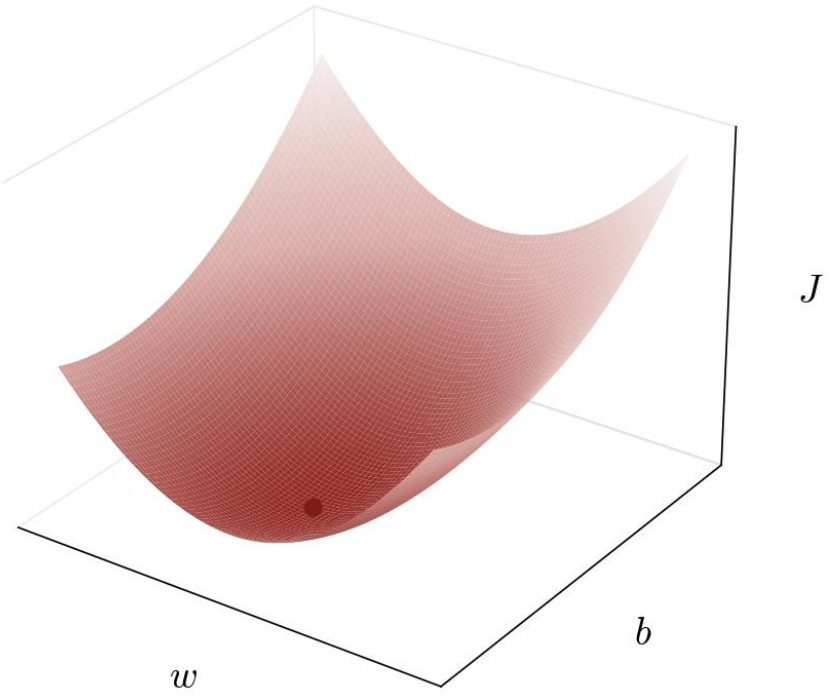
Why this loss? Maximum likelihood

$$P(y \mid \mathbf{x}) = \hat{y}^{\,y} \, (1 - \hat{y})^{1-y}$$

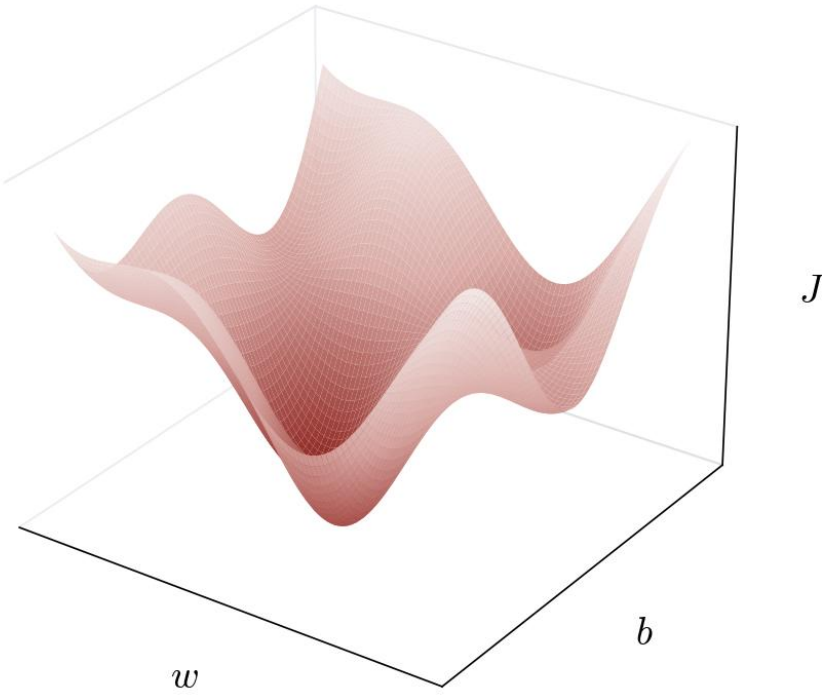
$$-\log P(y \mid \mathbf{x}) = -\big[\, y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \,\big] = \mathcal{L}(\hat{y}, y)$$

With i.i.d. examples, minimizing J is the same as **maximizing the likelihood** of the training labels.

The cost function is convex

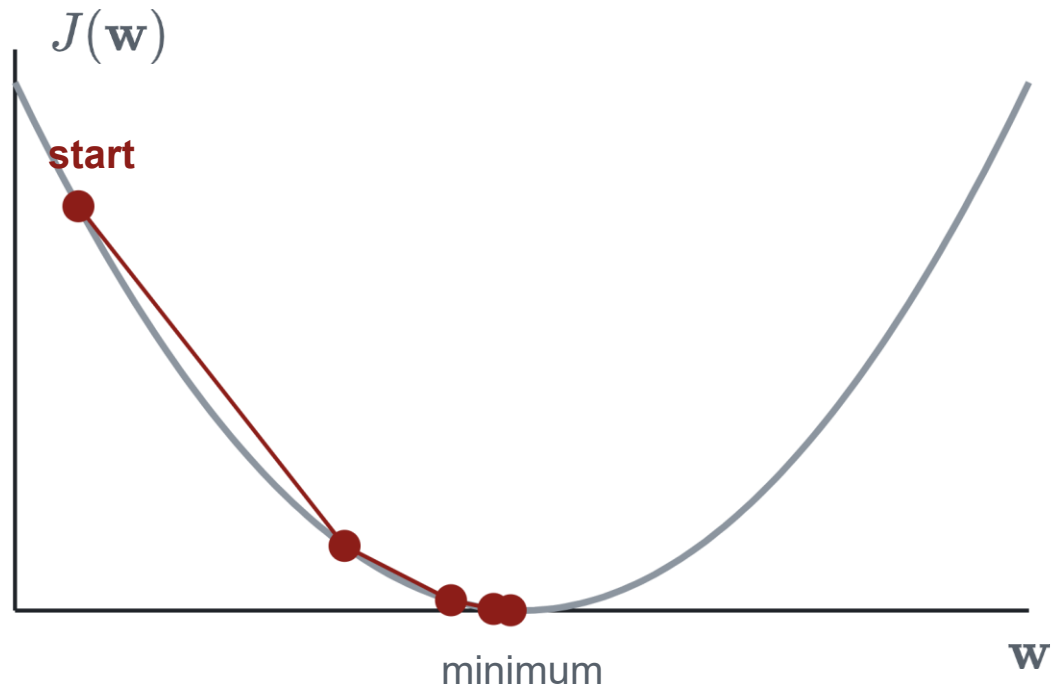


Logistic regression + cross-entropy: $J(\mathbf{w}, b)$ is convex, so gradient descent reaches the **global minimum** from any starting point.



Neural networks: J is non-convex.

Gradient descent



Repeat until convergence:

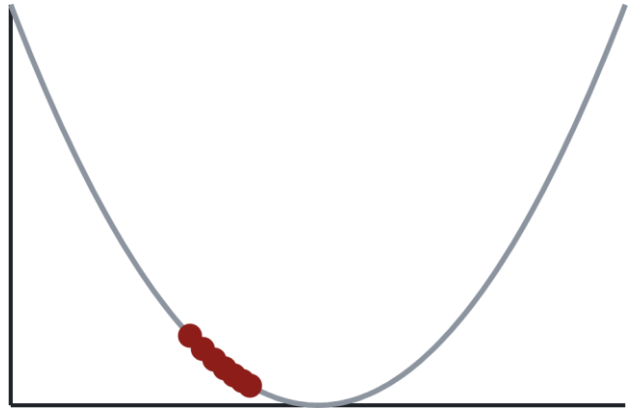
$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

$\alpha > 0$ is the **learning rate**. The derivative is the slope; we step **downhill**, against it.

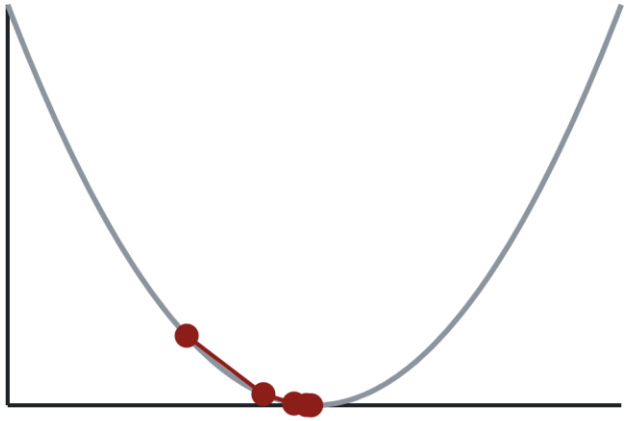
In the code we write dw for $\partial J / \partial \mathbf{w}$ and db for $\partial J / \partial b$

Choosing the learning rate



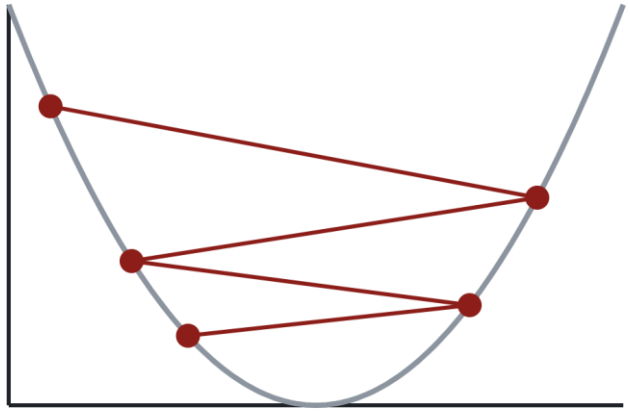
α too small

Tiny steps: training is very slow.



α about right

Reaches the minimum in a few steps.



α too large

Steps overshoot; the cost grows and training diverges.

> In practice: try values on a log scale, $\alpha \in \{0.001,\ 0.01,\ 0.1,\ 1\}$, and watch the cost curve.

A quick review of derivatives

Slope

$$f(a) = 3a$$

Nudge a by 0.001: f changes by 0.003.

$$\frac{df}{da} = 3 \quad \text{everywhere}$$

The slope can vary

$$f(a) = a^2$$

At $a = 2$: f goes from 4 to 4.004001.

$$\frac{df}{da} = 2a$$

Chain rule

If J depends on v, and v on a:

$$\frac{dJ}{da} = \frac{dJ}{dv}\frac{dv}{da}$$

The only calculus rule that backpropagation needs.

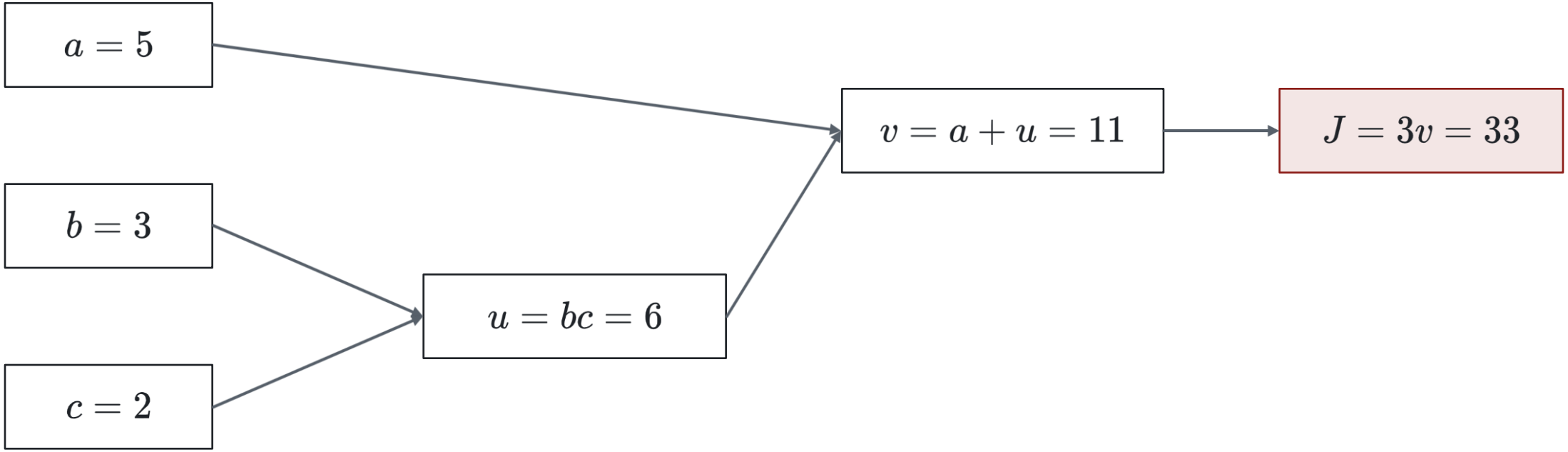
More derivative examples

Check each formula numerically: nudge a from 2 to 2.001 and compare $\Delta f / \Delta a$ with $f'(2)$.

$f(a)$	$\frac{df}{da}$	$f(2)$	$f(2.001)$	$\Delta f / \Delta a$	$f'(2)$
a^2	$2a$	4	4.004001	4.001	4
a^3	$3a^2$	8	8.012006	12.006	12
$\ln a$	$\frac{1}{a}$	0.693147	0.693647	0.4999	0.5
e^a	e^a	7.389056	7.396449	7.393	7.389

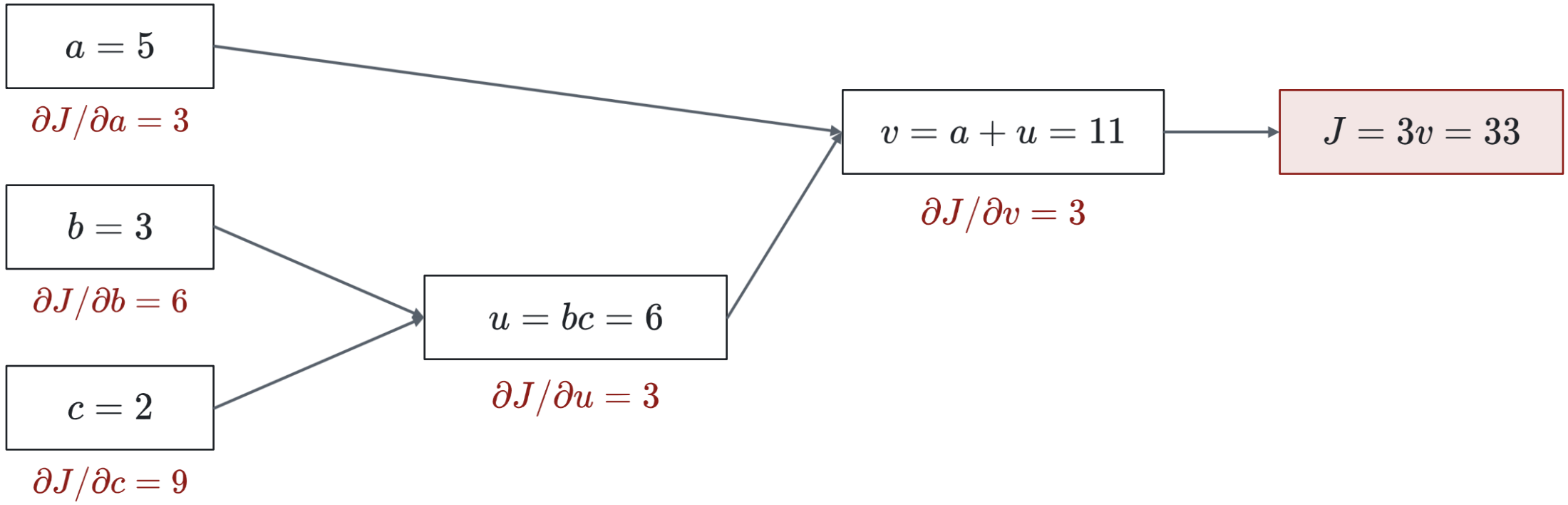
> The derivative is the slope of f at a point: a small nudge ε in a changes f by about $f'(a)\,\varepsilon$.

Computation graph: the forward pass



Evaluate $J(a, b, c) = 3(a + bc)$ step by step, **left to right**. Each node is one simple operation.

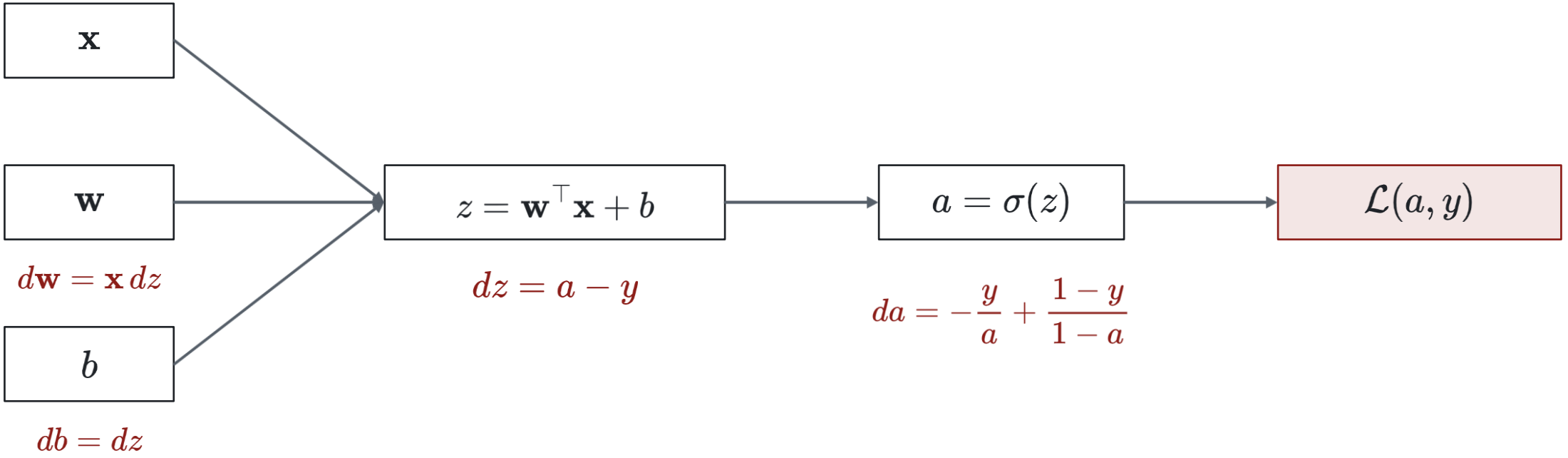
Computation graph: the backward pass



Go **right to left**, multiplying local derivatives: $\dfrac{\partial J}{\partial b} = \dfrac{\partial J}{\partial u}\dfrac{\partial u}{\partial b} = 3 \cdot c = 6.$

Logistic regression as a graph

$$z = w^T x + b$$
$$\hat{y} = a = \sigma(z)$$
$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$



Notation: $dz \equiv \partial \mathcal{L} / \partial z$, $d\mathbf{w} \equiv \partial \mathcal{L} / \partial \mathbf{w}$, and so on. $\mathbf{x}$ is data, so we never need its gradient.

Deriving dz = a − y

Chain rule through the two nodes $z \to a \to \mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a} \qquad \text{(loss)}$$

$$\frac{\partial a}{\partial z} = a\,(1-a) \qquad \text{(sigmoid)}$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a}\,\frac{\partial a}{\partial z} = -y\,(1-a) + (1-y)\,a = \boxed{a - y}$$

Since $z = \mathbf{w}^\top \mathbf{x} + b$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{x}\,(a-y), \qquad \frac{\partial \mathcal{L}}{\partial b} = a - y$$

Gradients over m examples

The cost is an average, so its gradient is an average too:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}^{(i)}\left(a^{(i)} - y^{(i)}\right), \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(a^{(i)} - y^{(i)}\right)$$

```
J = 0; dw = zeros(n); db = 0
for i in range(m):
    z = b
    for j in range(n):
        z += w[j] * x[i][j]
    a = sigmoid(z)
    J += loss(a, y[i])
    dz = a - y[i]
    for j in range(n):
        dw[j] += x[i][j] * dz
    db += dz
J /= m; dw /= m; db /= m
```

A direct implementation needs **two loops**: over the m examples and over the n features.

Vectorization: remove the loops

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{n} w_j \, x_j + b$$

Loop

```
z = 0
for j in range(n):
    z += w[j] * x[j]
z += b
```

Vectorized

```
z = x @ w + b
```

All m examples at once: $\mathbf{z} = \mathbf{X}\mathbf{w} + b, \qquad \mathbf{X} \in \mathbb{R}^{m \times n}, \; \mathbf{w} \in \mathbb{R}^{n \times 1} \Rightarrow \mathbf{z} \in \mathbb{R}^{m \times 1}$

Programming guideline: avoid explicit loops

Matrix–vector product

$$\mathbf{u} = A\mathbf{v}, \qquad u_i = \sum_j A_{ij} v_j$$

```
u = torch.zeros(n)
for i in range(n):
    for j in range(k):
        u[i] += A[i, j] * v[j]
```

Vectorized: u = A @ v

Element-wise functions

Maths	PyTorch
$u_i = e^{v_i}$	torch.exp(v)
$u_i = \log v_i$	torch.log(v)
$u_i = \lvert v_i \rvert$	torch.abs(v)
$u_i = \max(0, v_i)$	torch.clamp(v, min=0)
$u_i = v_i^2,\ 1/v_i$	v ** 2, 1 / v

Whenever possible, avoid explicit for-loops over examples or features: built-in operations run in parallel on CPU and GPU.

Logistic regression, fully vectorized

Computation	Shape	Meaning
$\mathbf{z} = \mathbf{X}\mathbf{w} + b$	$m \times 1$	linear part, all examples
$\mathbf{a} = \sigma(\mathbf{z})$	$m \times 1$	predictions (σ element-wise)
$d\mathbf{z} = \mathbf{a} - \mathbf{y}$	$m \times 1$	error of each example
$d\mathbf{w} = \frac{1}{m}\,\mathbf{X}^\top d\mathbf{z}$	$n \times 1$	same shape as $\mathbf{w}$
db = dZ.sum(dim=0, keepdim=True) / m	1×1	the mean of $d\mathbf{z}$
$\mathbf{w} := \mathbf{w} - \alpha\, d\mathbf{w}, \quad b := b - \alpha\, db$		one gradient descent step

> Check shapes at every step: $\mathbf{X}$ is $m \times n$ and $\mathbf{w}$ is $n \times 1$. A gradient always has the **same shape as its parameter**.

Broadcasting

$$\underbrace{\mathbf{X}\mathbf{w}}_{m\times 1} + \underbrace{b}_{\text{scalar}} \longrightarrow \mathbf{X}\mathbf{w} + b\,\mathbf{1}_m$$

PyTorch stretches b to shape $(m, 1)$ automatically: **broadcasting**.

Rule: compare shapes from the right; a dimension of size 1 is copied to match.

Operation ($\oplus \in \{+, -, \times, /\}$)	Result	What is copied
$(m, n) \oplus (1, n)$	(m, n)	the row, m times
$(m, n) \oplus (m, 1)$	(m, n)	the column, n times
$(m, 1) \oplus \text{scalar}$	$(m, 1)$	the scalar, m times

A common bug

If $\mathbf{y}$ has shape (m,) and $\mathbf{a}$ has shape (m, 1), then a − y has shape **(m, m)**, with no error message.

```
Y = Y.reshape(-1, 1)    # (m, 1)
```

Avoid rank-1 tensors of shape (m,): reshape to (m, 1) and check shapes.

Broadcasting: examples

Add a row to every row

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 100 & 200 & 300 \end{bmatrix} = \begin{bmatrix} 101 & 202 & 303 \\ 104 & 205 & 306 \end{bmatrix}$$

Add a column to every column

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 100 \\ 200 \end{bmatrix} = \begin{bmatrix} 101 & 102 & 103 \\ 204 & 205 & 206 \end{bmatrix}$$

In practice: centre each feature (column) of $\mathbf{X}$

$$\mathbf{X} = \begin{bmatrix} 2 & 4 \\ 4 & 8 \\ 6 & 6 \end{bmatrix}, \quad \boldsymbol{\mu} = \begin{bmatrix} 4 & 6 \end{bmatrix}, \quad \mathbf{X} - \boldsymbol{\mu} = \begin{bmatrix} -2 & -2 \\ 0 & 2 \\ 2 & 0 \end{bmatrix}$$

```
mu = X.mean(dim=0, keepdim=True)   # (1, n)
Xc = X - mu                        # (m, n)
```

keepdim=True keeps the shape (1, n), so the subtraction broadcasts over the rows.

The complete training loop

```
w = torch.zeros(n, 1); b = torch.zeros(1)
for it in range(num_iters):
    Z = X @ w + b                    # forward
    A = torch.sigmoid(Z)
    J = -(Y * torch.log(A)
          + (1 - Y) * torch.log(1 - A)).mean()
    dZ = A - Y                       # backward
    dw = X.T @ dZ / m
    db = dZ.mean()
    w -= lr * dw                     # update
    b -= lr * db
```

Forward

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b, \quad \mathbf{a} = \sigma(\mathbf{z})$$

Backward

$$d\mathbf{z} = \mathbf{a} - \mathbf{y}$$

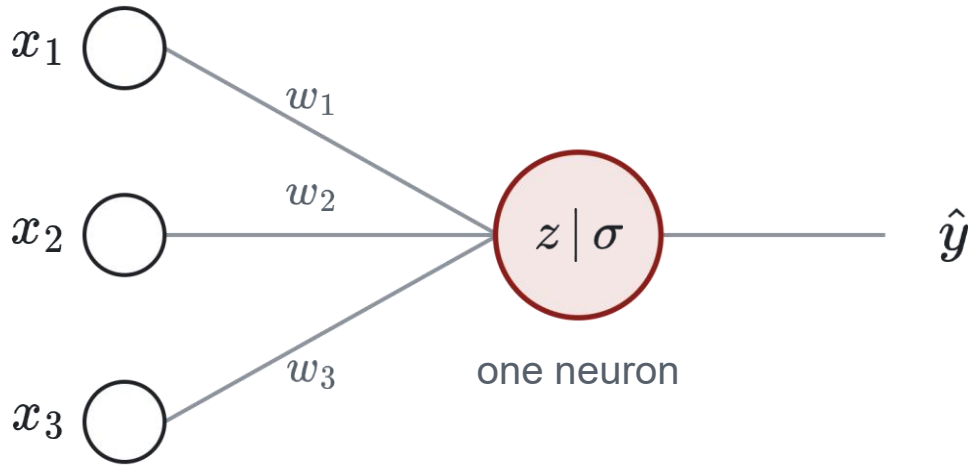
$$d\mathbf{w} = \frac{1}{m}\,\mathbf{X}^{\top} d\mathbf{z}, \quad db = \frac{1}{m}\,\mathbf{1}^{\top} d\mathbf{z}$$

Update

$$\mathbf{w} := \mathbf{w} - \alpha\, d\mathbf{w}, \quad b := b - \alpha\, db$$

Later: PyTorch autograd computes the backward pass for us, for any network.

Logistic regression = one neuron



A neuron computes a weighted sum, then a non-linearity:

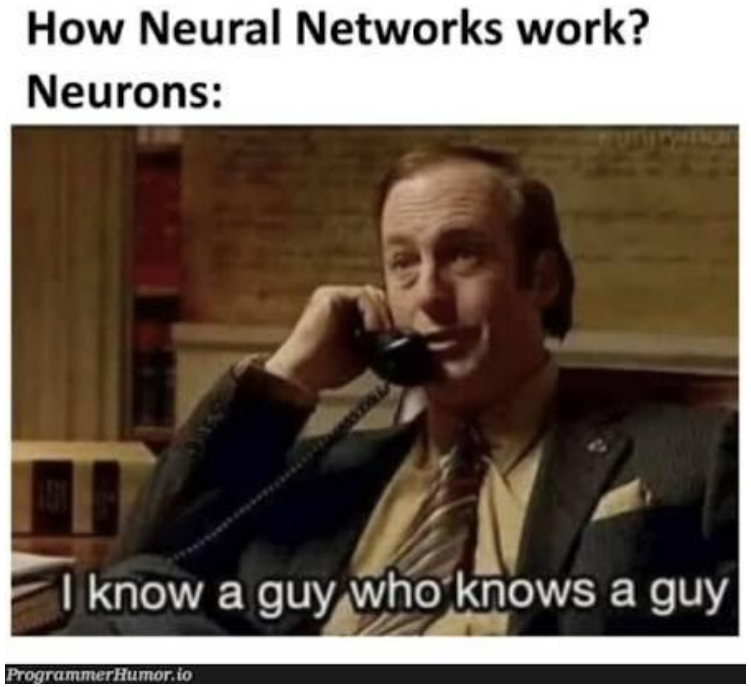
$$\hat{y} = \sigma\Big(\sum_{j=1}^{3} w_j x_j + b\Big)$$

Its decision boundary $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ is a **hyperplane**: a straight line in 2D.

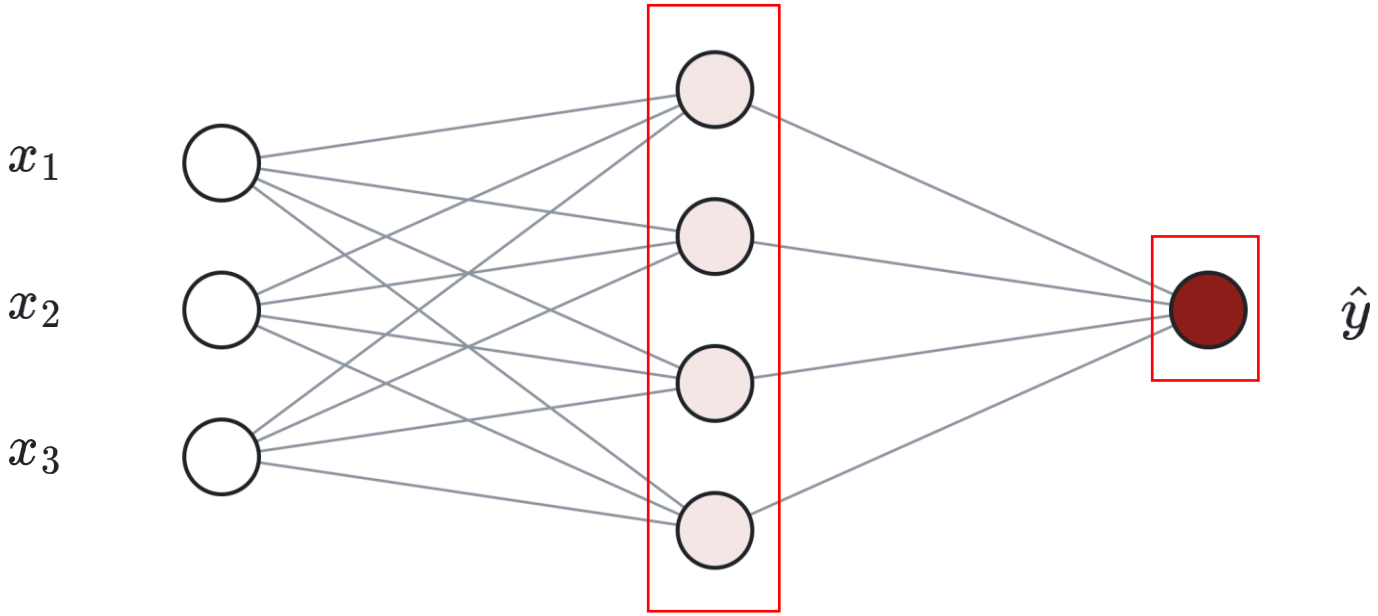
Next, in part 2: stack many neurons in layers.

Part 2

Shallow neural networks



From one neuron to a network



Input layer
$\mathbf{a}^{[0]} = \mathbf{x}$

Hidden layer
$\mathbf{a}^{[1]}$

Output layer
$a^{[2]} = \hat{y}$

Each circle is a neuron: a weighted sum, then an activation function g.

Hidden layer: its values are not in the training data; the network learns what they compute.

This is a **2-layer** network (the input layer is not counted), with parameters

$$\mathbf{W}^{[1]} \in \mathbb{R}^{3\times 4}, \quad \mathbf{b}^{[1]} \in \mathbb{R}^{1\times 4}$$
$$\mathbf{W}^{[2]} \in \mathbb{R}^{4\times 1}, \quad b^{[2]} \in \mathbb{R}$$

What each neuron computes

Hidden unit j of layer 1 is a logistic-regression unit with its own weights:

$$z_j^{[1]} = \mathbf{w}_j^{[1]\top}\mathbf{x} + b_j^{[1]}, \qquad a_j^{[1]} = g\big(z_j^{[1]}\big)$$

g is the **activation function**. The output layer does the same, with $\mathbf{a}^{[1]}$ as its input.

Notation	Meaning
$\square^{[l]}$	layer l
$\square^{(i)}$	training example i
$\square_j$	unit j of a layer
$n^{[l]}$	number of units in layer l, with $n^{[0]} = n$
$\mathbf{w}_j^{[1]}$	column j of $\mathbf{W}^{[1]} \in \mathbb{R}^{n^{[0]} \times n^{[1]}}$

Stacking the hidden units

For one example $\mathbf{x} \in \mathbb{R}^3$, the four hidden units of the previous slide, written as one row:

$$\underbrace{\begin{bmatrix} z_1^{[1]} & \cdots & z_4^{[1]} \end{bmatrix}}_{\mathbf{z}^{[1]}:\, 1\times 4} = \underbrace{\mathbf{x}^\top}_{1\times 3} \underbrace{\begin{bmatrix} | & & | \\ \mathbf{w}_1^{[1]} & \cdots & \mathbf{w}_4^{[1]} \\ | & & | \end{bmatrix}}_{\mathbf{W}^{[1]}:\, 3\times 4} + \underbrace{\begin{bmatrix} b_1^{[1]} & \cdots & b_4^{[1]} \end{bmatrix}}_{\mathbf{b}^{[1]}:\, 1\times 4}$$

Then $\mathbf{a}^{[1]} = g\big(\mathbf{z}^{[1]}\big)$, applied element-wise, and $z^{[2]} = \mathbf{a}^{[1]}\mathbf{W}^{[2]} + b^{[2]},\ \hat{y} = \sigma\big(z^{[2]}\big)$.

Column j of $\mathbf{W}^{[1]}$ is the weight vector of hidden unit j.

Vectorizing across examples

For loop over the m examples

$$\textbf{for } i = 1, \ldots, m:$$
$$\mathbf{z}^{[1](i)} = \mathbf{x}^{(i)\top}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$$
$$\mathbf{a}^{[1](i)} = g\big(\mathbf{z}^{[1](i)}\big)$$
$$z^{[2](i)} = \mathbf{a}^{[1](i)}\mathbf{W}^{[2]} + b^{[2]}$$
$$a^{[2](i)} = \sigma\big(z^{[2](i)}\big)$$

Vectorized

$$\mathbf{Z}^{[1]} = \mathbf{X}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$$
$$\mathbf{A}^{[1]} = g\big(\mathbf{Z}^{[1]}\big)$$
$$\mathbf{Z}^{[2]} = \mathbf{A}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}$$
$$\mathbf{A}^{[2]} = \sigma\big(\mathbf{Z}^{[2]}\big)$$

Stacking

$$\mathbf{A}^{[1]} = \begin{bmatrix} \text{--- } \mathbf{a}^{[1](1)} \text{ ---} \\ \vdots \\ \text{--- } \mathbf{a}^{[1](m)} \text{ ---} \end{bmatrix}$$

Rows: training examples. Columns: hidden units. $\mathbf{A}^{[1]} \in \mathbb{R}^{m \times n^{[1]}}$.

Row i of $\mathbf{X}\mathbf{W}^{[1]}$ is $\mathbf{x}^{(i)\top}\mathbf{W}^{[1]}$: the matrix product does all m examples at once, and $\mathbf{b}^{[1]}$ is broadcast over the rows.

Why we need non-linear activations

Suppose g is the identity, $g(z) = z$. Then

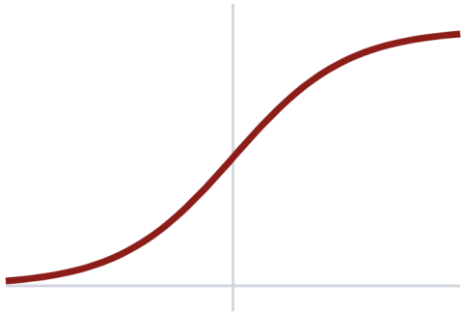
$$\mathbf{A}^{[2]} = \left(\mathbf{X}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}\right)\mathbf{W}^{[2]} + \mathbf{b}^{[2]}$$

$$= \mathbf{X}\underbrace{\mathbf{W}^{[1]}\mathbf{W}^{[2]}}_{\mathbf{W}'} + \underbrace{\mathbf{b}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}}_{\mathbf{b}'} = \mathbf{X}\mathbf{W}' + \mathbf{b}'$$

> A stack of linear layers is still **one linear layer**. Depth adds nothing without a non-linear g.

The one common place for a linear activation: the output of a regression model, e.g. predicting a price.

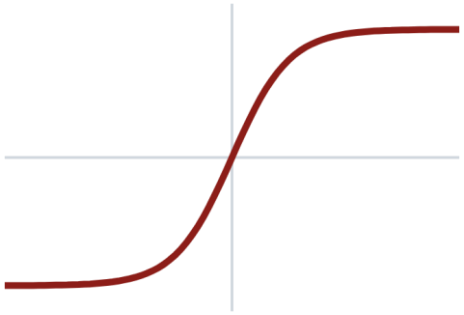
Common activation functions



Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

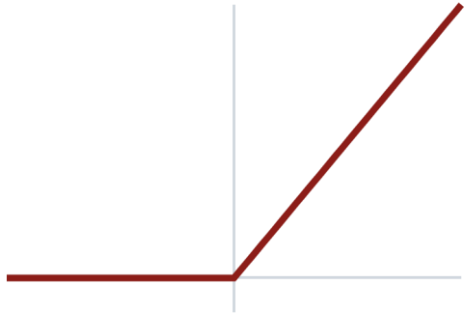
Range $(0, 1)$. Use for a binary output layer.



Tanh

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

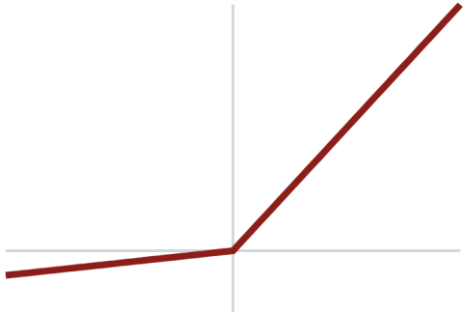
Range $(-1, 1)$, zero-centred. Usually better than σ in hidden layers.



ReLU

$$\max(0,\, z)$$

The default for hidden layers: cheap, gradient 1 for $z > 0$.



Leaky ReLU

$$\max(0.01z,\, z)$$

Small gradient for $z < 0$ (slope exaggerated in the plot).

Choosing an activation

Where	Use	Why
Hidden layers	ReLU (default)	Cheap to compute; trains well in deep networks
Hidden layers, if many units die	Leaky ReLU	Small gradient when $z < 0$
Binary output	Sigmoid σ	Output is a probability in $(0, 1)$
Multi-class output	Softmax	Probabilities over K classes
Regression output	Identity (linear)	Output can be any real number

> When in doubt, try a few choices on a dev set: deep learning is an empirical field.

Derivatives of the activations

Activation	$g(z)$	$g'(z)$, with $a = g(z)$	Check
Sigmoid	$\dfrac{1}{1+e^{-z}}$	$a\,(1-a)$	$g'(0) = \frac{1}{4},\ \ g'(\pm 10) \approx 0$
Tanh	$\tanh(z)$	$1-a^2$	$g'(0) = 1,\ \ g'(\pm 10) \approx 0$
ReLU	$\max(0, z)$	$\begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$	gradient never shrinks for $z > 0$
Leaky ReLU	$\max(0.01z, z)$	$\begin{cases} 0.01 & z < 0 \\ 1 & z > 0 \end{cases}$	never exactly zero

Gradient descent for neural networks

Parameters

$$\mathbf{W}^{[1]} \in \mathbb{R}^{n^{[0]} \times n^{[1]}}, \quad \mathbf{b}^{[1]} \in \mathbb{R}^{1 \times n^{[1]}}, \quad \mathbf{W}^{[2]} \in \mathbb{R}^{n^{[1]} \times n^{[2]}}, \quad \mathbf{b}^{[2]} \in \mathbb{R}^{1 \times n^{[2]}}$$

Cost

$$J\left(\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{W}^{[2]}, \mathbf{b}^{[2]}\right) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right), \qquad \hat{y} = a^{[2]}$$

repeat:

compute predictions $\hat{y}^{(i)}$, $i = 1, \ldots, m$ (forward)

$$d\mathbf{W}^{[l]} = \frac{\partial J}{\partial \mathbf{W}^{[l]}}, \quad d\mathbf{b}^{[l]} = \frac{\partial J}{\partial \mathbf{b}^{[l]}} \quad \text{(backward)}$$

$$\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha\, d\mathbf{W}^{[l]}, \quad \mathbf{b}^{[l]} := \mathbf{b}^{[l]} - \alpha\, d\mathbf{b}^{[l]} \quad (l = 1, 2)$$

Backward pass for a 2-layer network

Forward Pass

$$\mathbf{Z}^{[1]} = \mathbf{X}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$$
$$\mathbf{A}^{[1]} = g\left(\mathbf{Z}^{[1]}\right)$$
$$\mathbf{Z}^{[2]} = \mathbf{A}^{[1]}\mathbf{W}^{[2]} + \mathbf{b}^{[2]}$$
$$\mathbf{A}^{[2]} = \sigma\left(\mathbf{Z}^{[2]}\right)$$

Backward Pass

Gradient	Shape
$d\mathbf{Z}^{[2]} = \mathbf{A}^{[2]} - \mathbf{y}$	$m \times 1$
$d\mathbf{W}^{[2]} = \frac{1}{m}\,\mathbf{A}^{[1]\top} d\mathbf{Z}^{[2]}$	$n^{[1]} \times 1$
$d\mathbf{b}^{[2]} = \frac{1}{m}\,\mathbf{1}^{\top} d\mathbf{Z}^{[2]}$	1×1
$d\mathbf{A}^{[1]} = d\mathbf{Z}^{[2]}\,\mathbf{W}^{[2]\top}$	$m \times n^{[1]}$
$d\mathbf{Z}^{[1]} = d\mathbf{A}^{[1]} \odot g^{[1]\prime}\left(\mathbf{Z}^{[1]}\right)$	$m \times n^{[1]}$
$d\mathbf{W}^{[1]} = \frac{1}{m}\,\mathbf{X}^{\top} d\mathbf{Z}^{[1]}$	$n^{[0]} \times n^{[1]}$
$d\mathbf{b}^{[1]} = \frac{1}{m}\,\mathbf{1}^{\top} d\mathbf{Z}^{[1]}$	$1 \times n^{[1]}$

Random initialization

What goes wrong with zeros

- If $\mathbf{W}^{[1]} = \mathbf{0}$, then $a_1^{[1]} = a_2^{[1]} = \cdots$ for every input: all hidden units compute the same function.
- By symmetry $dz_1^{[1]} = dz_2^{[1]} = \cdots$, so the columns of $d\mathbf{W}^{[1]}$ are equal.
- After each update the columns of $\mathbf{W}^{[1]}$ are still equal. By induction the units stay identical: the **symmetry** problem.

The fix

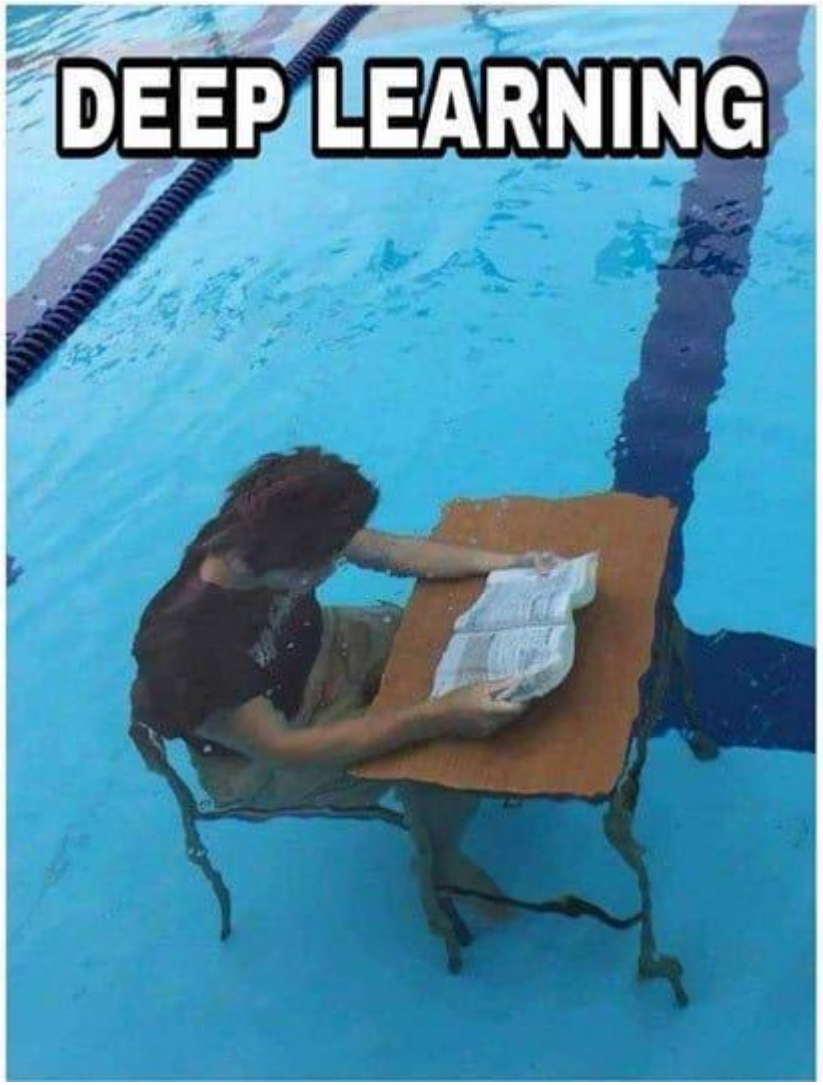
$$W_{jk}^{[1]} \sim \mathcal{N}\left(0,\ 0.01^2\right), \qquad \mathbf{b}^{[1]} = \mathbf{0}$$

```
W1 = torch.randn(n0, n1) * 0.01
b1 = torch.zeros(1, n1)
```

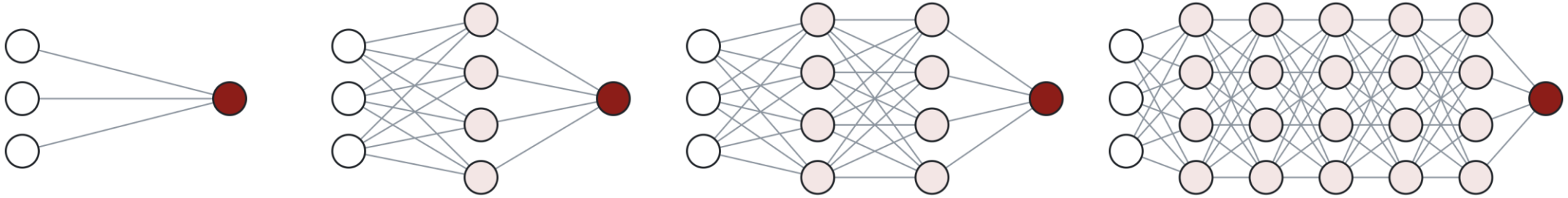
Why small? Large $|z|$ saturates σ and $\tanh$, so gradients vanish.

Part 3

Deep neural networks



What is a deep neural network?



Logistic regression

1 layer: shallow

1 hidden layer

2 layers

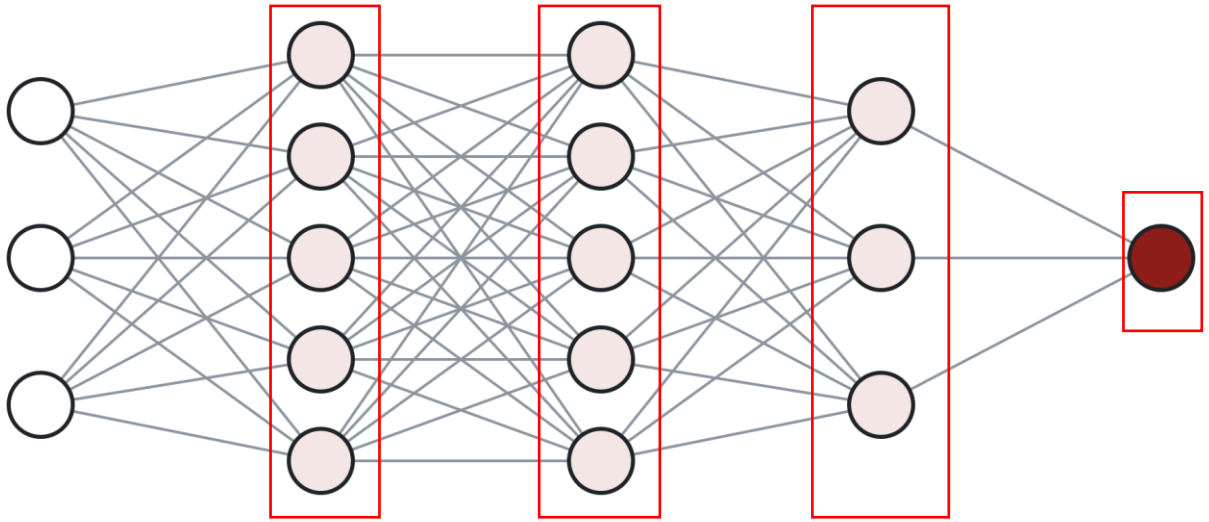
2 hidden layers

3 layers

5 hidden layers

6 layers: deep

Deep neural network notation



$n^{[0]} = 3$ $n^{[1]} = 5$ $n^{[2]} = 5$ $n^{[3]} = 3$ $n^{[4]} = 1$

- L = number of layers; here $L = 4$.
- $n^{[l]}$ = number of units in layer l, with $n^{[0]} = n_x = 3$ and $n^{[L]} = 1$.
- $\mathbf{a}^{[l]} = g^{[l]}\left(\mathbf{z}^{[l]}\right)$: activations of layer l; $\mathbf{a}^{[0]} = \mathbf{x}$, $a^{[L]} = \hat{y}$.
- $\mathbf{W}^{[l]}, \mathbf{b}^{[l]}$: the parameters that compute $\mathbf{z}^{[l]}$.

Forward propagation in a deep network

$$
\begin{aligned}
&\mathbf{A}^{[0]} = \mathbf{X} \\
&\textbf{for } l = 1, \ldots, L: \\
&\quad \mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]} \\
&\quad \mathbf{A}^{[l]} = g^{[l]}\left(\mathbf{Z}^{[l]}\right) \\
&\hat{\mathbf{y}} = \mathbf{A}^{[L]}
\end{aligned}
$$

```
A = X
for W, b, g in layers:
    Z = A @ W + b
    A = g(Z)
Y_hat = A
```

The loop over **layers** is fine: each layer needs the output of the previous one, so it cannot be vectorized away.

Inside each layer there is no loop over examples or units.

Getting the matrix dimensions right

Example: $L = 5$ with layer sizes $n^{[0]}, \ldots, n^{[5]} = 2, 3, 5, 4, 2, 1$.

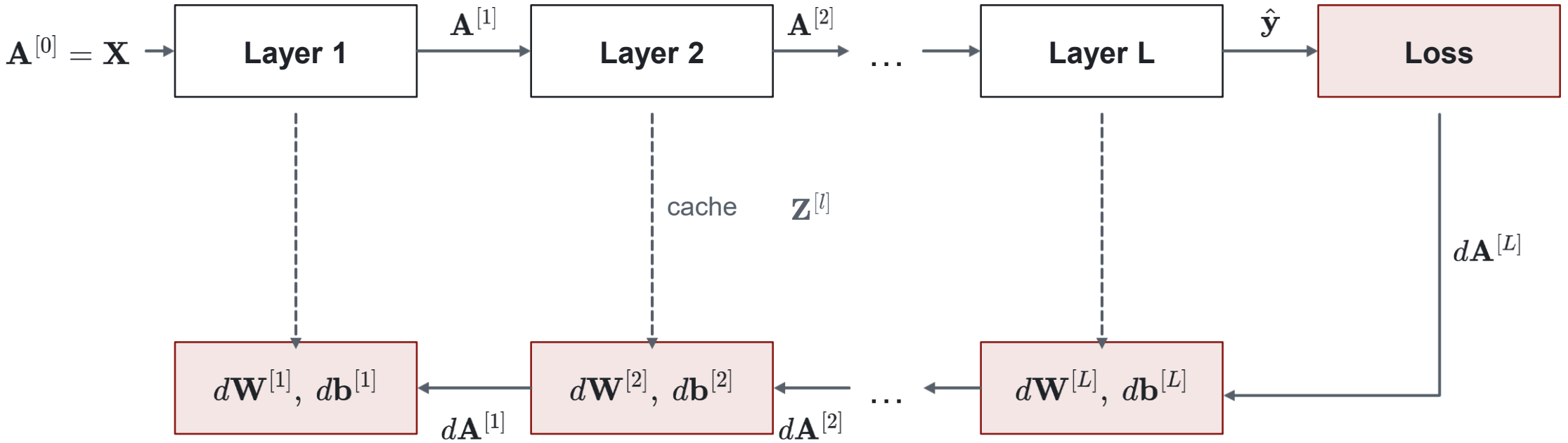
l	$\mathbf{W}^{[l]}$	$\mathbf{b}^{[l]}$	$\mathbf{Z}^{[l]}, \mathbf{A}^{[l]}$
1	2×3	1×3	$m \times 3$
2	3×5	1×5	$m \times 5$
3	5×4	1×4	$m \times 4$
4	4×2	1×2	$m \times 2$
5	2×1	1×1	$m \times 1$

The general rule

$$\mathbf{W}^{[l]},\ d\mathbf{W}^{[l]} : n^{[l-1]} \times n^{[l]}$$
$$\mathbf{b}^{[l]},\ d\mathbf{b}^{[l]} : 1 \times n^{[l]}$$
$$\mathbf{Z}^{[l]},\ \mathbf{A}^{[l]},\ d\mathbf{Z}^{[l]},\ d\mathbf{A}^{[l]} : m \times n^{[l]}$$

Checking these shapes catches most bugs before you run any code.

Forward and backward, layer by layer



Forward: layer l maps $\mathbf{A}^{[l-1]} \mapsto \mathbf{A}^{[l]}$ and caches $\mathbf{Z}^{[l]}$. **Backward:** it maps $d\mathbf{A}^{[l]} \mapsto d\mathbf{A}^{[l-1]}$ and outputs $d\mathbf{W}^{[l]}, d\mathbf{b}^{[l]}$.

The equations for layer l

Forward

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}\left(\mathbf{Z}^{[l]}\right)$$

Input $\mathbf{A}^{[l-1]}$; output $\mathbf{A}^{[l]}$; cache $\mathbf{Z}^{[l]}$ and $\mathbf{A}^{[l-1]}$.

Backward

$$d\mathbf{Z}^{[l]} = d\mathbf{A}^{[l]} \odot g^{[l]\prime}\left(\mathbf{Z}^{[l]}\right)$$

$$d\mathbf{W}^{[l]} = \frac{1}{m}\,\mathbf{A}^{[l-1]\top} d\mathbf{Z}^{[l]}$$

$$d\mathbf{b}^{[l]} = \frac{1}{m}\,\mathbf{1}^{\top} d\mathbf{Z}^{[l]}$$

$$d\mathbf{A}^{[l-1]} = d\mathbf{Z}^{[l]}\,\mathbf{W}^{[l]\top}$$

Start of the backward pass, for a sigmoid output with cross-entropy:

$$d\mathbf{A}^{[L]} = -\frac{\mathbf{Y}}{\mathbf{A}^{[L]}} + \frac{1-\mathbf{Y}}{1-\mathbf{A}^{[L]}} \quad \Rightarrow \quad d\mathbf{Z}^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y}$$

Divisions are element-wise. Typical network: ReLU in layers $1, \ldots, L-1$ and a sigmoid in layer L.

Parameters and hyperparameters

Parameters

Learned by gradient descent:

$$\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \ \mathbf{W}^{[2]}, \mathbf{b}^{[2]}, \ \ldots, \ \mathbf{W}^{[L]}, \mathbf{b}^{[L]}$$

Hyperparameters

Chosen by you; they control the parameters:

- learning rate α and number of iterations
- number of layers L and units per layer $n^{[l]}$
- activation functions $g^{[l]}$
- later: batch size, regularization, momentum

> Finding good values is empirical: idea → code → experiment, repeated.

More than two classes: softmax

Output layer with K units, one per class. For logits $\mathbf{z} \in \mathbb{R}^K$:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K$$

Every $\hat{y}_k > 0$ and $\sum_k \hat{y}_k = 1$: a probability distribution over the K classes.

Class k	z_k	e^{z_k}	$\hat{y}_k$
1	5	148.4	0.842
2	2	7.4	0.042
3	-1	0.4	0.002
4	3	20.1	0.114
Sum		176.3	1.000

Loss for K classes

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

$\mathbf{y}$ is **one-hot**: $y_c = 1$ for the true class c and 0 elsewhere, so $\mathcal{L} = -\log \hat{y}_c$.

Example from the previous slide, true class 1: $\mathcal{L} = -\log 0.842 \approx 0.17$.

Gradient of softmax + cross-entropy: $d\mathbf{Z}^{[L]} = \hat{\mathbf{Y}} - \mathbf{Y}$ (the same form as sigmoid + binary cross-entropy)

> In PyTorch, nn.CrossEntropyLoss takes the **logits** $\mathbf{z}$ and the class index, and applies softmax itself: do not add a softmax layer before it.

A deep network in PyTorch

```
model = nn.Sequential(
    nn.Flatten(),                # 28×28 → 784
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 64), nn.ReLU(),
    nn.Linear(64, 10))           # 10 logits
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for X, y in train_loader:
    loss = loss_fn(model(X), y)  # forward
    loss.backward()              # backward
    opt.step()                   # update
    opt.zero_grad()
```

Same structure as today's equations:

- nn.Linear(784, 128) computes $\mathbf{Z} = \mathbf{A}\mathbf{W} + \mathbf{b}$ with $\mathbf{W} \in \mathbb{R}^{784\times128}$ (PyTorch stores $\mathbf{W}^\top$ as .weight).
- $L = 3$ layers, ReLU hidden units, softmax inside the loss.
- loss.backward() runs backpropagation through every layer.
- opt.zero_grad() clears the gradients, because PyTorch adds them up by default.

Summary

- Logistic regression $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ is a one-neuron network, trained by minimizing the cross-entropy cost J.
- Backpropagation is the chain rule on a computation graph, from right to left; for a sigmoid output, $dz = a - y$.
- Vectorize: no loops over examples or units; check shapes at every layer.
- Hidden layers learn their own features; a non-linear g is essential (ReLU by default).
- Layer l: $\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$, $\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$. Random initialization; softmax + cross-entropy for K classes.

Next (weeks 3–5): improving deep neural networks: data splits, bias and variance, regularization, optimization, tuning, and structuring machine learning projects.